

REPORT DI ANALISI MALWARE

Titolo: Analisi di un eseguibile sospetto rinominato `W32.Nimda.Amm.exe`

Autore: Akatsuki

Data: 07/09/2026

Strumenti utilizzati: Wireshark, Terminale/PowerShell (calcolo hash), VirusTotal, ANY.RUN, Detect It Easy.

1. Executive Summary

È stato analizzato un file eseguibile sospetto, denominato `W32.Nimda.Amm.exe`, intercettato durante il monitoraggio del traffico di rete e trasferito via HTTP. Il nome suggeriva una possibile correlazione con il noto worm W32.Nimda.

Esito dell'analisi: il file non è il worm W32.Nimda e non presenta indicatori statici o dinamici di malware. Si tratta con ogni probabilità di un eseguibile Windows non malevolo, rinominato con il nome di un malware storico. Non si configura quindi un "falso positivo" in senso antivirale — poiché nessun motore AV lo ha rilevato — ma piuttosto un falso allarme dovuto al nome fuorviante.

2. Fase 1 – Analisi del traffico di rete (Wireshark)

Obiettivo: identificare la natura del file durante il transito sulla rete e ricavare indicatori preliminari.

Metodologia:

È stata utilizzata la funzione *Follow TCP Stream* di Wireshark per ispezionare il contenuto della richiesta HTTP GET `/W32.Nimda.Amm.exe`.

Risultati:

L'ispezione del flusso TCP ha mostrato dati binari non stampabili, intervallati da stringhe leggibili. Le stringhe identificate includono:

- `MZ` e `PE`: firme standard di un eseguibile Windows (Portable Executable).

- `.text`, `.rdata`, `.data`, `.rsrc`, `.reloc`: nomi delle sezioni standard di un file PE.
- `KERNEL32.dll`, `NTDLL.DLL`, `msvcrt.dll`: librerie di sistema Windows.
- `WINBRAND.dll`: libreria standard di Windows (`C:\Windows\System32\winbrand.dll`), usata per le informazioni di branding del sistema operativo. Non è una libreria anomala e non indica di per sé un framework per installer.

Nota tecnica:

`NTDLL.DLL` è una DLL nativa di basso livello, normalmente non importata direttamente dalle applicazioni user-mode. La sua presenza non è di per sé sospetta, ma è meno comune rispetto a `KERNEL32.dll` o `msvcrt.dll`.

3. Fase 2 – Fingerprinting e intelligence (hash e VirusTotal)

Obiettivo: ottenere l'impronta univoca del file e confrontarla con i database globali.

Metodologia:

Dopo aver esportato l'oggetto HTTP da Wireshark (*File* → *Export Objects* → *HTTP*), è stato calcolato l'hash crittografico SHA-256.

Risultati:

- SHA-256:
`db06c3534964e3fc79d2763144ba53742d7fa250ca336f4a0fe724b75aaff386`
- VirusTotal: 0/71 rilevamenti. Nessun motore antivirus commerciale ha classificato il file come dannoso.

Interpretazione:

Il file non possiede firme statiche di malware conosciuto. Questo dato è coerente con l'ipotesi di un eseguibile non malevolo.

4. Fase 3 – Analisi dinamica (ANY.RUN)

Obiettivo: eseguire il file in un ambiente isolato per osservarne il comportamento reale.

Metodologia:

Il campione è stato caricato ed eseguito sulla sandbox pubblica ANY.RUN, monitorando processi, traffico di rete e modifiche al sistema.

Risultati e osservazioni:

Il traffico di rete osservato nella sandbox è attribuibile a processi di sistema Windows, compatibili con una macchina virtuale appena avviata:

- `svchost.exe` (PID 9100/9108): traffico verso `login.live.com` e server OCSP/CRL (`ocsp.digicert.com`, `crl.microsoft.com`) per la verifica dei certificati. Gli errori `400 Bad Request` sono comuni in ambienti sandbox isolati.
- `MoUsoCoreWorker.exe` (PID 5276): processo di Windows Update, ha contattato `settings-win.data.microsoft.com`. La VM era configurata per bloccare gli aggiornamenti, reindirizzando il traffico verso l'host fittizio `neverupdatewindows10.com`, tecnica comune nei laboratori di analisi.
- `SIHClient.exe` (PID 6096): servizio di manutenzione di Windows Update.

Non è stata osservata alcuna connessione dannosa direttamente attribuibile al processo `W32.Nimda.Amm.exe`. Il file non ha generato traffico verso C2, non ha scaricato payload aggiuntivi e non ha mostrato comportamenti malevoli evidenti.

Limiti dell'analisi dinamica:

Il report originale non includeva il process tree completo, né eventuali modifiche a file system o registro. L'assenza di comportamento malevolo è quindi supportata dai dati disponibili, ma non può essere considerata esaustiva senza un'analisi più approfondita.

5. Analisi statica aggiuntiva (Detect It Easy – opzionale)

Non è stato possibile utilizzare Detect It Easy nel dettaglio, ma l'analisi delle stringhe e delle importazioni non ha evidenziato packer o offuscamenti tipici di malware. Il file appare come un normale eseguibile PE.

6. Conclusioni finali

L'analisi a più livelli porta alle seguenti conclusioni:

1. Contenuto: la struttura del file (PE, sezioni standard, importazioni) è compatibile con un eseguibile Windows legittimo. `WINBRAND.dll` è una libreria standard di Windows e non costituisce un indicatore di anomalia.
2. Firma: il file non è riconosciuto come dannoso da nessun motore antivirus (0/71 su VirusTotal).
3. Comportamento: in ambiente sandbox non ha prodotto traffico di rete malevolo né azioni sospette. Il traffico osservato è interamente generato da servizi di sistema Windows.
4. Nome: `W32.Nimda.Amm.exe` è solo un'etichetta, probabilmente un nome di rilevamento AV o un nome assegnato manualmente. Non corrisponde al worm Nimda.

Conclusione: il file non è il worm W32.Nimda e non è pericoloso. Si tratta di un falso allarme dovuto al nome fuorviante, non di un falso positivo antivirale.

7. Raccomandazioni

- Chiudere l'allarme: il file può essere considerato non malevolo.
- Non giudicare un file dal nome: analizzare sempre hash, firma digitale e comportamento.
- Verificare la firma digitale: se disponibile, controllare la firma del codice per confermare la legittimità dell'eseguibile.
- Migliorare l'analisi dinamica: includere process tree, modifiche a file system e registro, e un baseline del traffico di sistema.
- Documentare i limiti: distinguere sempre tra "nessun indicatore rilevato" e "assenza dimostrata di comportamento malevolo".

1. Perché `W32.Nimda.Amm.exe` è l'unico file nella cattura?

Perché la cattura di rete è stata filtrata o limitata a quella specifica transazione HTTP. L'analista ha probabilmente acquisito solo il traffico relativo al download di quel file (es. con un filtro `http.request.uri contains "W32.Nimda.Amm.exe"`), oppure il

pcap fornito per l'esercizio conteneva esclusivamente quella richiesta GET e la relativa risposta. Non significa che sulla rete ci fosse solo quel file, ma che nella cattura esaminata era l'unico oggetto HTTP presente.

2. Cosa sono tutti quei simboli mostrati nella finestra *Follow TCP Stream*? Sono rumore di connessione? Dati? Spiega.

Non sono rumore di connessione: sono dati binari.

La finestra *Follow TCP Stream* di Wireshark mostra il payload TCP grezzo. Poiché il file scaricato è un eseguibile (binario), la maggior parte dei byte non corrisponde a caratteri ASCII stampabili. Wireshark li visualizza come punti, quadratini o simboli. Si tratta quindi del contenuto reale del file eseguibile (il body della risposta HTTP), non di rumore. Il rumore di connessione sarebbe semmai rappresentato da ritrasmissioni, ACK, ecc., che non appaiono in quella vista.

3. Ci sono alcune parole leggibili sparse tra i simboli. Perché sono lì?

Perché all'interno del file PE (Portable Executable) sono presenti stringhe ASCII che fanno parte della struttura del file stesso. Ad esempio:

- MZ, PE: firme del formato eseguibile.
- .text, .rdata, .data, .rsrc, .reloc: nomi delle sezioni.
- KERNEL32.dll, NTDLL.DLL, msvcrt.dll, WINBRAND.dll: nomi delle librerie importate.

Queste stringhe sono leggibili perché il formato PE le memorizza come testo. Wireshark le mostra correttamente quando i byte corrispondono a caratteri stampabili.

4. Domanda Sfida: quale eseguibile è realmente?

Dai frammenti visibili — in particolare `WINBRAND.dll` — si può dedurre che il file rinominato `W32.Nimda.Amm.exe` è in realtà `winver.exe` (il visualizzatore della versione di Windows).

Motivazione:

- `WINBRAND.dll` è una libreria standard di Windows usata da `winver.exe` per mostrare le informazioni di branding del sistema operativo (es. “Windows 10 Pro”).
- Le sezioni PE standard e le importazioni (`KERNEL32.dll`, `NTDLL.DLL`, `msvcrt.dll`) sono compatibili con un piccolo eseguibile di sistema come `winver.exe`.
- Non è un installer, come ipotizzato erroneamente nel report originale: `WINBRAND.dll` non è anomala.

Quindi il file è semplicemente una copia di `winver.exe` rinominata con il nome di un malware storico.

5. Nel processo di analisi del malware, quale sarebbe un probabile passo successivo per un analista di sicurezza?

Dopo aver stabilito che il file è un `winver.exe` rinominato e non malevolo, i passi successivi sono:

1. Verifica dell'hash: confrontare lo SHA-256 con database noti (VirusTotal, NSRL) e con l'hash di `winver.exe` di un'installazione Windows pulita.
2. Controllo della firma digitale: verificare se il file è firmato digitalmente da Microsoft (anche se il rename può invalidare la firma? In realtà la firma è nel file, non nel nome, quindi dovrebbe rimanere valida).
3. Analisi statica approfondita: usare PESTudio, Detect It Easy, `strings` per confermare che non ci siano modifiche sospette.
4. Analisi dinamica controllata: eseguire in sandbox (ANY.RUN, Cuckoo) monitorando processi, file system, registro e rete.
5. Documentazione e chiusura dell'allarme: registrare che si tratta di un falso

allarme dovuto al nome fuorviante, non di un falso positivo antivirale.

6. Indagine sul contesto: capire perché il file è stato rinominato e trasferito (errore umano, test, tentativo di depistaggio?).
7. Aggiornamento delle regole di detezione: evitare alert basati solo sul nome del file, privilegiando hash e comportamento.

In sintesi: il file è innocuo, ma l'analista deve sempre verificare l'integrità e il contesto prima di chiudere il caso.



